DECENTRALIZED BOUNTY ESCROW AND TOURNAMENT SYSTEM FOR OPEN-SOURCE SOFTWARE

BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE AND ENGINEERING

Case Study Report

submitted by

BHUKHA LAKSHAMN SINGH NAIK
24505A0502

Under the guidance of
Mr. A. Prashant, Assistant Professor

Department of Computer Science and Engineering
Prasad V Potluri Siddhartha Institute of Technology
(Permanently affiliated to JNTU-Kakinada, Approved by AICTE)
(An NBA & NAAC accredited and ISO 9001:2015 certified institute)

Kanuru, Vijayawada - 520 007
Academic Year 2026-2027

Prasad V Potluri Siddhartha Institute of Technology
(Permanently affiliated to JNTU-Kakinada, Approved by AICTE)
(An NBA & NAAC accredited and ISO 9001:2015 certified institute)
Kanuru, Vijayawada - 520 007

CERTIFICATE

This is to certify that the case study report entitled **"Decentralized Bounty Escrow and Tournament System for Open-Source Software"** is submitted by **Bhukha Lakshamn Singh Naik (24505A0502)**, Department of Computer Science and Engineering, as **Assignment 2** for the **Blockchain Technology (23CS4701)** course in **4-1** during the academic year **2026-2027**, under the guidance of Mr. A. Prashant, Assistant Professor.

The work presented in this report describes an educational prototype and its observed implementation boundaries. The report does not claim production deployment, a security audit, or reported test results.

Course Coordinator
Mr. A. Prashant
Assistant Professor,
Department of CSE,
PVPSIT, Vijayawada

Head of the Department
Dr. A. Jayalakshmi,
Professor and Head,
Department of CSE,
PVPSIT, Vijayawada

MARKS
ASSIGNMENT-2: ____ / 5

INDEX

S. No.	Contents	Page No.
1	Abstract	1
2	Introduction	2
3	Background	3
4	Blockchain Basics	5
5	Use Case and System Design	7
6	Implementation	13
7	Benefits, Challenges, and Limitations	24
8	Conclusion and Future Work	27
9	SDGs Addressed	28
10	References and Links	29
11	Appendix A	30

ABSTRACT

The project is an educational prototype for escrow-backed bounties and tournaments for open-source software. A sponsor (the host) deploys a Solidity contract, MergeXHub, built and tested with Foundry and using OpenZeppelin's ReentrancyGuard. Developers bind their GitHub handle to an Ethereum wallet during a Setup phase, and the host then locks an ETH prize pool in the contract. A Python oracle built with web3.py and requests polls the GitHub REST API endpoint for the latest release of a repository. When the release tag equals the configured target tag, the oracle submits the release author's GitHub handle to settleMilestoneWinner, and the contract pays the full prize pool to the wallet registered for that handle in the same transaction that marks the event Completed. If no settlement happens before the deadline, the host can call refund. The contract also contains a second mode, ContributionWeight, in which the oracle pushes scores and names the final winner.

The prototype has clear limits: it relies on one trusted oracle and one trusted host, nothing proves that a wallet owner controls the GitHub handle they register, the oracle script implements the milestone-race flow only, prize splitting and a frontend are not built, the system has been described for a local Anvil chain only, and no test results, gas measurements, or independent audit are reported. It is suitable only for demonstrations with test funds.

Keywords: blockchain, smart contract, escrow, oracle, GitHub API, bounty, Solidity, Foundry

1. INTRODUCTION

1.1. Blockchain Overview

A blockchain is a shared ledger in which transactions are grouped into blocks and linked using cryptographic hashes. In a network with multiple independent participants, consensus rules help participants agree on an accepted history. The design can make later alteration detectable and costly, but the strength of that property depends on the network, its consensus mechanism, and its governance [1]. Smart-contract platforms such as Ethereum extend the ledger with program code that holds and moves value according to fixed rules [2].

1.2. Introduction to the Project

This project explores how a smart contract can act as a neutral escrow between people who fund open-source work and developers who deliver it. The funds sit in a contract vault, the evidence of delivery is a public GitHub event (a published release), and an off-chain oracle carries that evidence to the chain. A contract cannot call GitHub by itself, so the oracle is the bridge between the two worlds. The implementation is a prototype for local development with Foundry's Anvil node and a mock GitHub release endpoint; it is not a deployed service.

1.3. Organization of the Report

Chapter 2 describes the problems of paying for open-source work and how far the project addresses them. Chapter 3 introduces the blockchain concepts used. Chapter 4 presents the use case, objectives, architecture, implementation status, on-chain state, and contract interface. Chapter 5 explains the implementation, including excerpts of the systemHub.sol, the oracle script, the main workflows, and testing. Chapter 6 discusses benefits, challenges, and limitations. Chapter 7 concludes and lists future work, Chapter 8 relates the project to selected Sustainable Development Goals, Chapter 9 lists the references, and Appendix A gives the repository link and source-file pointers.

2. BACKGROUND

Open-source software is valuable but often poorly funded, and paying for it is awkward. Sponsors fear paying for work that is never delivered, developers fear delivering work that is never paid, and maintainers spend time checking work and arranging payment. A smart contract can hold the money in escrow and apply fixed payout rules, but it cannot by itself decide whether work was delivered, and it cannot judge code quality. Sections 2.1 to 2.7 take the main challenges in turn, and Section 2.8 summarizes how far the project addresses each of them.

2.1. Trust Between Sponsor and Developer

Without a neutral holder of funds, one side must trust the other: the sponsor pays first and risks non-delivery, or the developer works first and risks non-payment. The system locks the prize pool in the contract before the competition is active, so the sponsor cannot quietly withdraw it while the event is Active except through the deadline-gated refund, and the developer can see that the funds exist. The contract follows fixed rules, but it is the host who deploys it and chooses the oracle, so the host still decides who is trusted (Section 3.6).

2.2. Centralized Bounty Platforms

Centralized bounty platforms hold the money, charge fees, and can freeze accounts, so they are a single point of failure and control. The system replaces the platform with a contract whose payout rules are public code. This removes the platform as a custodian but not the need to trust the oracle that reports results; the trust moves rather than disappears.

2.3. Objective Evidence of Delivery

Pull requests, merges, and release tags are public, timestamped GitHub events and make better evidence than a private judgment. The system uses the simplest of these: the tag name and the author login of the latest release returned by the GitHub releases endpoint. A matching tag shows that a release with that name was published; it does not show that the code is correct, secure, or the best contribution.

2.4. Identity: GitHub Handle and Wallet

The blockchain recognizes addresses, while GitHub reports logins. The system bridges them with an on-chain registry in which a developer calls registerDeveloper with their handle during Setup. Each handle and each wallet can be registered only once. However, the contract cannot check that the caller actually owns the GitHub account, so a person could register someone else's handle with their own wallet before that person does (Section 6.2). Verified linking is listed as future work.

2.5. Protection if Nothing Is Delivered

Funds must not be stuck if no one wins or if the oracle disappears. MergeXHub stores a deadline at deployment, and after it passes the host can call refund while the event is still Active.

The deadline protects the sponsor, but it does not stop an oracle settlement made before the host refunds (Section 5.3.6).

2.6. Auditability

Contract events let observers reconstruct what happened. MergeXHub emits DeveloperRegistered, PrizeFunded, ScoreUpdated, WinnerSettled, and RefundIssued (Section 5.3.7). No dashboard or API consumes these events in this repository, so auditing currently means reading logs from the chain directly.

2.7. Dependence on the Oracle

The oracle is the most trusted component: it decides which handle wins. The contract checks only that the caller is the oracle address and that the named handle is registered. It does not re-check GitHub, and in Weight mode it does not compare scores before paying the winner the oracle names. Multi-oracle or dispute designs are future work (Chapter 7).

2.8. Summary of the System's Position

Table 2.1 summarizes the challenges discussed above and how far the project addresses each of them.

Table 2.1: Open-source bounty challenges and the position of the system

Challenge	Issue	Position of the system
Sponsor-developer trust	One side must pay or work first	ETH locked in the contract before the event is Active; payout rules fixed in code
Centralized platforms	Custody, fees, account freezing	No platform custody; trust moves to the host and the oracle
Evidence of delivery	Work must be verified objectively	Latest release tag and author read from the GitHub API; does not judge quality
Identity	Wallet and GitHub login are different identities	Write-once handle registry in the Setup phase; ownership not verified
Fund safety	Funds must not be stuck	Host refund after deadline while Active
Auditability	Actions should be reconstructable	Five events emitted; no dashboard
Oracle trust	One reporter decides the winner	Single oracle address; no multi-oracle or dispute window

3. BLOCKCHAIN BASICS

3.1. Decentralization and Consensus

Decentralization distributes control and ledger maintenance among independent participants, and consensus is the process by which they agree on the valid ledger state. Public networks can reduce reliance on a single operator when participants run separate nodes. The system in its development setup does not have this property: it uses a local Anvil node, which is convenient for fast testing but provides no independent consensus. The design assumes an EVM-compatible chain, and the design document names a public testnet (Sepolia) as a later target; no testnet deployment is evidenced in the repository.

3.2. Immutability

Immutability means that recorded data cannot be altered or removed without detection under the network rules. In MergeXHub the oracle, kind, host, and deadline are declared immutable, so they are fixed at deployment and cannot be changed afterwards, including by the host. The contract has no function to change the oracle address, which is safe against replacement but means a lost or leaked oracle key cannot be rotated. Contract state such as status, prize pool, and registered wallets does change as the event progresses, and a local development chain can be discarded by whoever controls it.

3.3. Transparency and Metadata

On a public chain, anyone can read contract state and logs. MergeXHub stores GitHub handles, wallet addresses, scores, and the prize pool, and the githubToWallet and scores mappings are public getters. This improves auditability, but it links a public GitHub identity to a wallet address permanently for anyone who reads the registry.

3.4. Smart Contracts

A smart contract is program code executed by a blockchain when a transaction or read call invokes its functions. MergeXHub uses Solidity ^0.8.24 and restricts state-changing functions by role: the host funds and refunds, the oracle settles and updates scores, and developers register. Solidity 0.8 reverts on arithmetic overflow, and the contract uses custom errors, which are cheaper than revert strings [3].

3.5. Hashing and Digital Signatures

Ethereum uses keccak256 as its hash function. MergeXHub's mappings are keyed by string handles; Solidity derives each storage slot from a keccak256 hash of the key, so the contract does not compare strings directly. An earlier prototype of the project compared release tags by hashing both strings with keccak256, because Solidity has no native string equality. Transactions are authorized by digital signatures: the oracle script signs each settlement transaction with its private key, and the contract recognizes the signer through msg.sender. A leaked key lets anyone act as the oracle.

3.6. The Oracle Pattern

Smart contracts are deterministic and isolated, so they cannot make HTTP requests. An oracle is an off-chain service that fetches external data and submits it in a transaction [4]. In this system the oracle is a Python script whose address is fixed in the contract at deployment. This makes the oracle the highest-trust component: the contract trusts whatever handle the oracle submits, so a compromised or dishonest oracle can direct the prize to any registered handle.

3.7. Escrow and Safe Payout

Escrow holds funds until conditions are met. MergeXHub pays by sending ETH with a low-level call after updating its own state (status set to Completed and prizePool set to zero before the transfer), and it applies OpenZeppelin's nonReentrant modifier to every function that sends ETH. This follows the checks-effects-interactions pattern recommended against reentrancy attacks [5]. If the transfer fails the whole transaction reverts and the state change is undone.

4. USE CASE AND SYSTEM DESIGN

4.1. Problem Statement and Objectives

Sponsors and developers of open-source work lack a neutral way to agree on payment: platforms are centralized, and direct payment requires trust. The project addresses this with an escrow contract and an oracle that reads public GitHub release data. The objectives are:

1. Escrowed prize: the host locks an ETH prize pool in the contract before the event becomes Active.
2. Identity registry: developers bind a GitHub handle to a wallet once, during Setup.
3. Automatic settlement: when the oracle sees the target release tag, it names the release author as winner and the contract pays that author's wallet.
4. Two competition modes: a Milestone Race (first to publish the target release) and a Contribution Weight tournament (oracle-pushed scores and an oracle-named winner).
5. Sponsor protection: the host can reclaim the prize pool after the deadline if the event is still Active.
6. Safe contract practices and honest documentation: custom errors, events, nonReentrant, checks-effects-interactions, and a clear statement of which features are complete, partial, or planned.

4.2. Proposed Solution and User Roles

The proposed solution is a hybrid application: GitHub is the source of evidence, a Python oracle is the bridge, and a Solidity contract is the vault and registry. The participants are:

- **Host / Sponsor:** deploys MergeXHub, chooses the oracle address, mode, and duration, funds the prize pool once, and may refund after the deadline.
- **Developer:** registers a GitHub handle with their wallet during Setup and receives the prize if named winner.
- **Oracle:** the account whose address is stored in the contract; it settles the Race winner, pushes scores, and finalizes Weight tournaments. The oracle is a trusted component, not a neutral party.
- **Maintainer:** publishes the release on GitHub that serves as the evidence; this role exists outside the contract.
- **Observer:** anyone who reads the contract state and events.

The design progressed through three prototypes, summarized in Table 4.0 from the project's design document.

Table 4.0: Evolution of the project contracts

Phase	Contract	What it demonstrated
1	SimpleEscrow	Locked funds with sponsor release and time-locked refund
2	SimpleBounty	An oracle triggers payment when a release tag matches
3	AdvancedBountyHub	Multiple developers and a handle-to-wallet registry
Current	MergeXHub	Race and Weight modes, write-once registry, deadline refund, events, custom errors, reentrancy guard

Outside the scope of the prototype are verified identity linking, prize splitting, multi-oracle settlement, a dispute process, fiat or token payments, a frontend, mainnet deployment, and an independent audit.

4.3. System Architecture

Figure 4.1 shows the architecture of the system. Developers and the host interact with the contract directly through transactions; the oracle is the only component that reads GitHub and then writes settlement results to the chain.

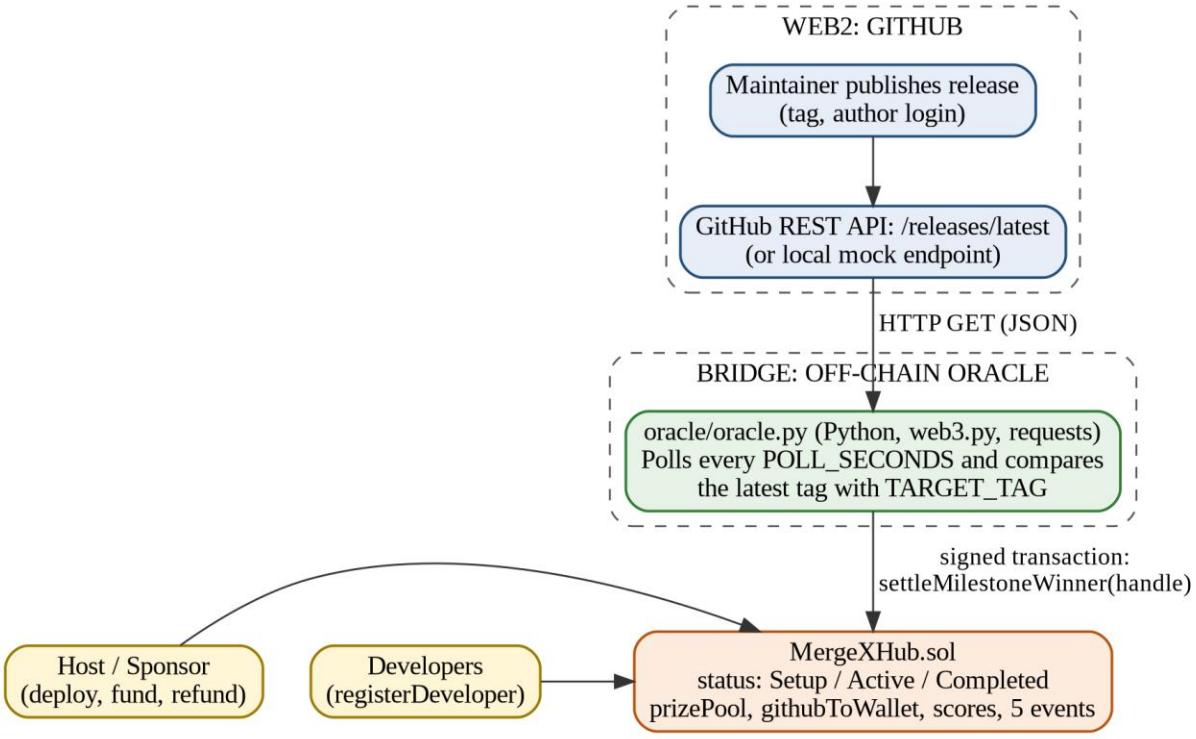


Figure 4.1: System Architecture of the system.

4.3.1. GitHub Data Source

The oracle calls the releases endpoint of the GitHub REST API for a repository given as owner/repo and reads two fields: the tag name and the login of the release author. The base URL

is configurable through the GITHUB_API variable, and the repository contains a mock/repos/demo/demo/releases directory laid out like that endpoint, which allows the demonstration to run without the live GitHub service. A personal access token can be supplied to raise rate limits.

4.3.2. Oracle

The oracle is a single Python script (oracle/oracle.py) configured entirely through environment variables: the RPC URL, the oracle private key, the hub address, the repository, the target tag, an optional token, and the polling interval (default 5 seconds). It loads the contract ABI from the Foundry build output, so forge build must run first.

4.3.3. Blockchain Layer

MergeXHub is compiled and tested with Foundry; the local chain for demonstrations is Anvil. The contract imports OpenZeppelin's ReentrancyGuard from the lib directory (a git submodule listed in .gitmodules). Roles are stored as immutable addresses at deployment: the host is the deploying account, and the oracle address is a constructor argument.

4.3.4. Project Layout

The repository root holds the contract source (src), the oracle (oracle), tests (test), the mock release data (mock), a demonstration script (demo.sh), Foundry configuration (foundry.toml and foundry.lock), a documentation folder (docs), and the design the design document.

4.4. Implementation Status of Features

Table 4.1 classifies each feature from the source code. Implemented means present and connected end to end in the files read; Partial means part exists but a necessary part is missing; Planned means absent from the code and proposed as future work in Chapter 7.

Table 4.1: Implementation status of the system features

Feature	Status	Basis in the source code
Prize escrow with one-time funding (Setup to Active)	Implemented	fundPrizePool: onlyHost, onlySetup, reverts on zero value
Write-once GitHub handle and wallet registry, Setup only	Implemented	registerDeveloper reverts with HandleTaken or WalletTaken
Milestone Race settlement by oracle	Implemented	settleMilestoneWinner plus oracle.py
Host refund after deadline	Implemented	refund: onlyHost, onlyActive, deadline check
Events, custom errors, nonReentrant, checks-effects-interactions	Implemented	Five events, twelve errors, guard on all payout functions
Python oracle polling the releases endpoint	Implemented	oracle.py (Race mode only)
Weight mode: score update and finalize	Partial	Contract functions exist; the oracle script does not compute or push scores
Winner chosen by highest score	Planned	finalizeWeightTournament pays the handle the oracle names; scores are not compared

Feature	Status	Basis in the source code
Automated tests and demo script	Sources present	test directory and demo.sh exist; results not reported (Section 5.6)
Verified identity linking (signed gist)	Planned	Described in the design document only
Prize splitting (1st/2nd/3rd) and tie rules	Planned	Single winner takes the whole pool
Multi-oracle or dispute window; oracle key rotation	Planned	One immutable oracle address
Frontend dashboard; public testnet deployment	Planned	No frontend source in the folders; Sepolia named in the design document

4.5. On-Chain State Design

MergeXHub keeps all state on-chain and uses no database. Table 4.2 lists the state variables and Table 4.3 shows how data is divided between GitHub, the oracle process, and the blockchain.

Table 4.2: State variables of the systemHub

Variable	Type	Rules and purpose
oracle	address, immutable	The only account allowed to settle, update scores, and finalize
kind	enum Kind, immutable	MilestoneRace or ContributionWeight, fixed at deployment
host	address, immutable	The deployer; funds and refunds
deadline	uint256, immutable	block.timestamp at deployment plus the duration in seconds
status	enum Status	Setup, Active, or Completed
prizePool	uint256	Set once by fundPrizePool; reset to zero on payout or refund
githubToWallet	mapping(string to address payable), public	Handle to payout wallet
_walletRegistered	mapping(address to bool), private	Prevents one wallet registering two handles
scores	mapping(string to uint256), public	Scores pushed by the oracle in Weight mode

Table 4.3: Placement of the system data

Location	Data held
GitHub	Repository, release tag, release author login
Oracle process	RPC URL, oracle private key, hub address, target tag, token (environment variables); ABI read from the Foundry build output
Blockchain (Anvil in the demo)	Roles, deadline, status, prize pool, handle-to-wallet registry, scores, and the five events

4.6. Contract Interface

Table 4.4 lists the externally callable functions of the systemHub and who may call them.

Table 4.4: External functions of the systemHub

Function	Caller	Allowed when	Purpose
constructor(oracle, kind, durationSeconds)	Host	Deployment	Fix roles, mode, and deadline; status = Setup
registerDeveloper(handle)	Any wallet	Setup	Bind a handle to the caller's wallet, once
fundPrizePool() payable	Host	Setup	Lock ETH and set status to Active
settleMilestoneWinner(handle)	Oracle	Active, Race mode	Pay the registered wallet and complete
updateScore(handle, score)	Oracle	Active, Weight mode	Store a score for a registered handle
finalizeWeightTournament(winnerHandle)	Oracle	Active, Weight mode	Pay the named registered handle and complete
refund()	Host	Active, at or after the deadline	Return the prize pool to the host and complete

Every state-changing call is checked first by role (NotHost, NotOracle), then by status (NotSetup, NotActive) and mode (WrongKind), and the payout functions are protected by nonReentrant.

Figure 4.2 shows how these checks fit into the life cycle of one event. The status moves only forward, from Setup to Active to Completed, and no function reopens a Completed event.

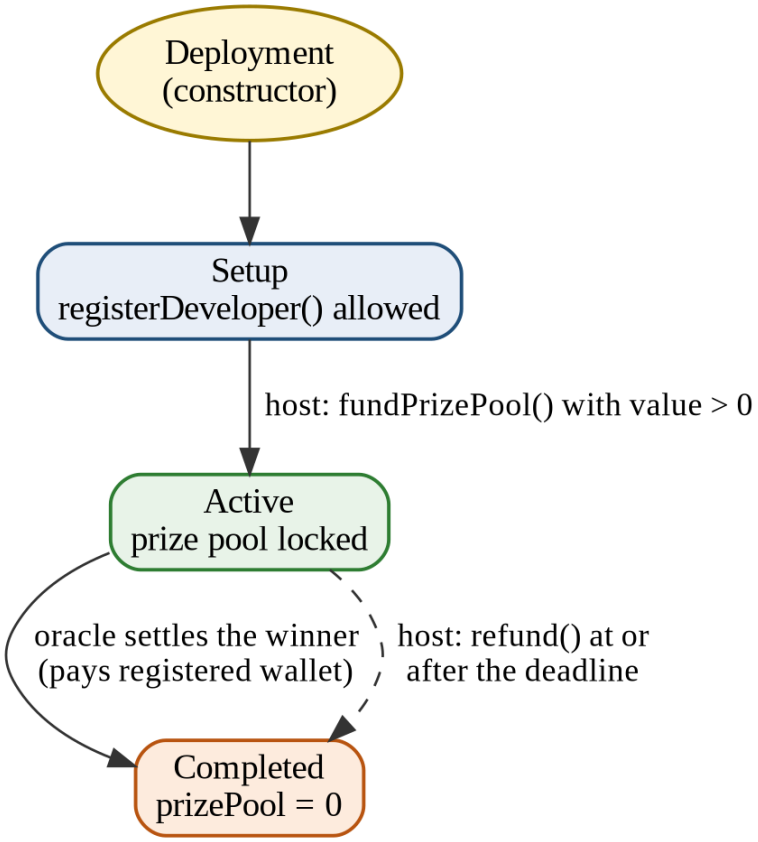


Figure 4.2: Status transitions of MergeXHub.

5. IMPLEMENTATION

5.1. Environment and Tools

The repository combines a Solidity contract built with Foundry, a Python oracle, and a shell demonstration script. Exact versions are declared in the project files [9]. Table 5.1 lists the technology stack.

Table 5.1: Technology stack of the system

Component	Technology	Role
Smart contract	Solidity ^0.8.24 [3]	Escrow, registry, settlement, refund
Contract libraries	OpenZeppelin Contracts, ReentrancyGuard [5]	Reentrancy protection on payout functions
Build and test	Foundry (forge) [6]	Compile and run the test suite
Local chain	Anvil [6]	Local EVM node for the demonstration
Oracle	Python 3.10+, web3.py [7], requests	Poll GitHub, sign and send settlement transaction
Data source	GitHub REST API, releases endpoint [8]	Latest release tag and author login
Demonstration	Bash script (demo.sh) and mock release data	End-to-end local run described in the README

5.2. Oracle Implementation

The oracle (oracle/oracle.py) is a single script that runs a polling loop. It reads its configuration from environment variables (RPC_URL, ORACLE_PRIVATE_KEY, HUB_ADDR, REPO, TARGET_TAG, and optionally GITHUB_API, GH_TOKEN, and POLL_SECONDS) and loads the contract ABI from the Foundry artifact out/MergeXHub.sol/MergeXHub.json; if the file is missing it logs an error and exits, so forge build must run first.

The function latest_release requests the releases endpoint for the repository with an optional bearer token and a 15-second timeout, and returns the tag name and the author login of the latest release. The main loop calls it every POLL_SECONDS seconds. If the tag equals TARGET_TAG, the script calls settle with the author login: it builds a settleMilestoneWinner transaction from the oracle account, signs it, sends it, waits up to 60 seconds for the receipt, exits with failure if the receipt status is not 1, and otherwise prints the transaction hash and exits successfully. Network errors are logged and retried on the next poll; a ContractLogicError (for example a revert) is logged and ends the script with failure.

The script handles the Milestone Race flow only. It does not compute contribution scores, call updateScore, or call finalizeWeightTournament. It checks only the latest release, so if a newer release replaces the target release before a poll, the target tag is never seen. GitHub logins are case-insensitive while the contract compares handles as exact strings, so a registered handle must match the login's capitalization exactly.

Figure 5.1 summarizes the control flow of the oracle script described above (simplified; the handling of a contract revert is omitted).

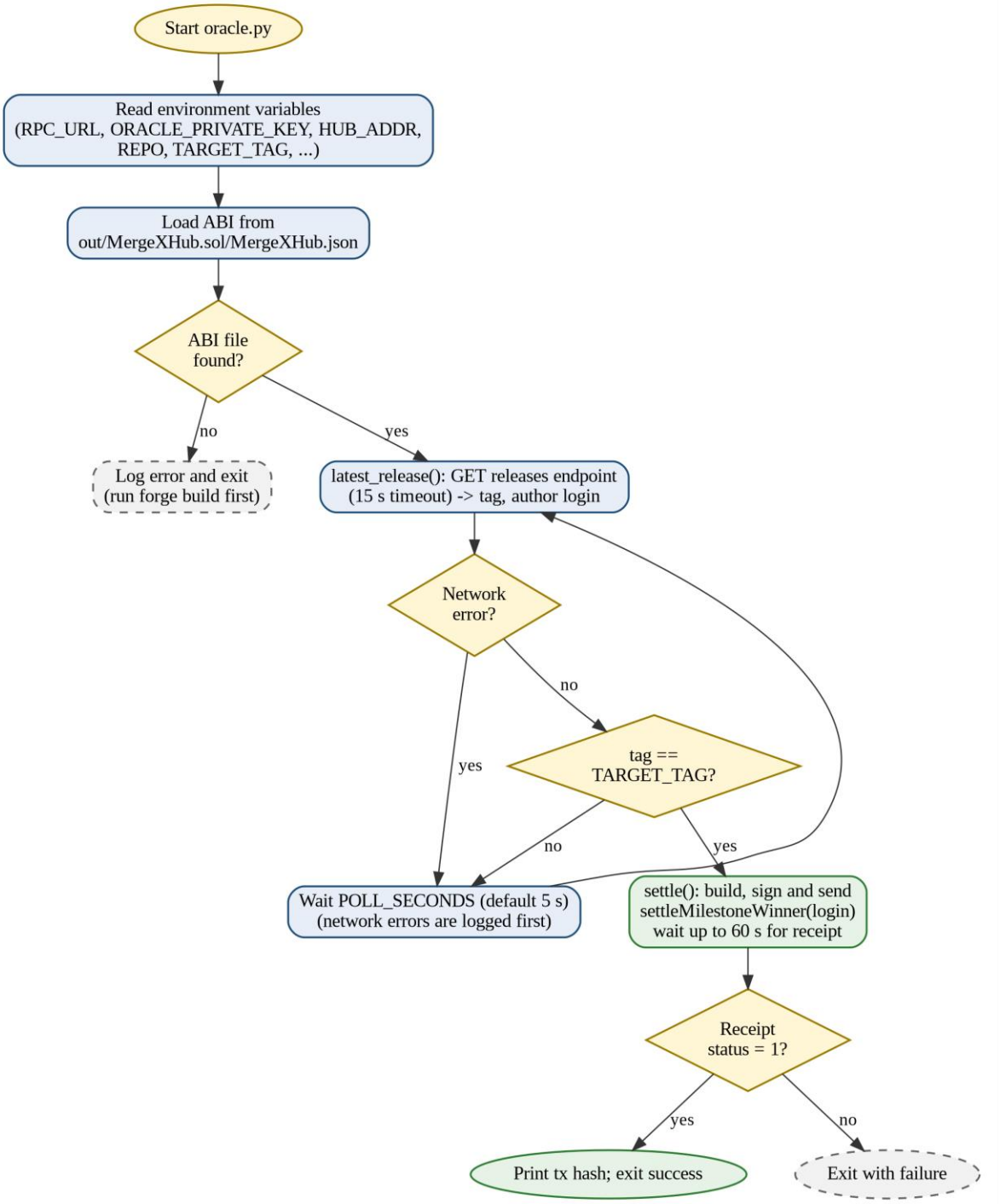


Figure 5.1: Control flow of the oracle script (oracle/oracle.py).

5.3. Smart Contract Implementation

This section presents excerpts of src/MergeXHub.sol copied from the project source. Some comments and spacing are removed, and each listing is an excerpt: statements between excerpts are not reproduced. Names, modifiers, and checks are unchanged. The excerpts describe what the code does; they are not evidence that it has been tested or audited.

5.3.1. Declaration, State, and Modifiers

The contract inherits OpenZeppelin's ReentrancyGuard and declares two enums (Kind and Status), twelve custom errors, and five events. Four values are immutable: oracle, kind, host, and deadline. The deadline is the deployment time plus the duration passed to the constructor in seconds. Four modifiers guard the functions: onlyHost, onlyOracle, onlySetup, and onlyActive. The error AlreadyFunded is declared but never used, because the onlySetup check already stops a second funding.

```
contract MergeXHub is ReentrancyGuard {
    enum Kind   { MilestoneRace, ContributionWeight }
    enum Status { Setup, Active, Completed }

    address public immutable oracle;
    Kind    public immutable kind;
    address public immutable host;
    uint256 public immutable deadline;

    Status  public status;
    uint256 public prizePool;

    mapping(string  => address payable) public githubToWallet;
    mapping(address => bool)            private _walletRegistered;
    mapping(string  => uint256)         public  scores;

    constructor(address _oracle, Kind _kind, uint256 _durationSeconds) {
        oracle   = _oracle;
        kind     = _kind;
        host     = msg.sender;
        deadline = block.timestamp + _durationSeconds;
        status   = Status.Setup;
    }

    modifier onlyHost()   { if (msg.sender != host)   revert NotHost();   _; }
    modifier onlyOracle() { if (msg.sender != oracle) revert NotOracle(); _; }
    modifier onlySetup()  { if (status != Status.Setup)  revert NotSetup();  _; }
    modifier onlyActive() { if (status != Status.Active) revert NotActive(); _; }
```

Listing 5.1: Contract declaration, state, constructor, and modifiers (MergeXHub.sol, excerpt)

The constructor does not check that the oracle address is non-zero or different from the host.

5.3.2. Developer Registration

registerDeveloper binds the caller's wallet to a GitHub handle (Listing 5.2). It works only in the Setup phase, reverts with HandleTaken if the handle is already bound, and reverts with WalletTaken if the wallet is already used for another handle. The binding is therefore write-once

on both sides. The function does not check that the handle is non-empty, and it cannot prove that the caller owns the GitHub account.

```
function registerDeveloper(string calldata handle) external onlySetup {
    if (githubToWallet[handle] != address(0))  revert HandleTaken();
    if (_walletRegistered[msg.sender])          revert WalletTaken();
    githubToWallet[handle]        = payable(msg.sender);
    _walletRegistered[msg.sender] = true;
    emit DeveloperRegistered(handle, msg.sender);
}
```

Listing 5.2: Handle registration - registerDeveloper (excerpt)

5.3.3. Funding the Prize Pool

fundPrizePool lets only the host fund, only during Setup, and only with a non-zero value (Listing 5.3). It stores the amount, moves the status to Active, and emits PrizeFunded. Because the status leaves Setup, funding can happen only once and registration closes at the same moment; the host decides when that happens.

```
function fundPrizePool() external payable onlyHost onlySetup {
    if (msg.value == 0) revert ZeroValue();
    prizePool = msg.value;
    status    = Status.Active;
    emit PrizeFunded(msg.sender, msg.value);
}
```

Listing 5.3: Funding - fundPrizePool (excerpt)

5.3.4. Milestone Race Settlement

settleMilestoneWinner is callable only by the oracle, only while Active, and only in MilestoneRace mode (Listing 5.4). It looks up the wallet for the handle and reverts with UnregisteredHandle if there is none. It then sets the status to Completed and the pool to zero before emitting WinnerSettled and sending the ETH with a low-level call, which follows the checks-effects-interactions order. If the transfer fails the function reverts and the state change is undone. The contract does not check that the deadline has not passed, so the oracle can still settle after the deadline until the host calls refund.

```
function settleMilestoneWinner(string calldata handle)
    external onlyOracle onlyActive nonReentrant
{
    if (kind != Kind.MilestoneRace) revert WrongKind();
    address payable winner = githubToWallet[handle];
    if (winner == address(0)) revert UnregisteredHandle();

    uint256 amount = prizePool;
    status    = Status.Completed;
    prizePool = 0;

    emit WinnerSettled(handle, winner, amount);

    (bool ok,) = winner.call{value: amount}("");
    if (!ok) revert TransferFailed();
}
```

Listing 5.4: Race settlement - settleMilestoneWinner (excerpt)

5.3.5. Contribution Weight Mode

In ContributionWeight mode the oracle calls updateScore to store a score for a registered handle, which emits ScoreUpdated, and later calls finalizeWeightTournament with the winner's handle (Listing 5.5). The finalize function applies the same checks and payout steps as Race settlement. The contract does not compare scores: it pays the handle named by the oracle, so the stored scores are informational and the winner decision is entirely the oracle's. This replaces the earlier prototype's on-chain leader tracking, which could become wrong if scores were corrected.

```
function updateScore(string calldata handle, uint256 score)
    external onlyOracle onlyActive
{
    if (kind != Kind.ContributionWeight) revert WrongKind();
    if (githubToWallet[handle] == address(0)) revert UnregisteredHandle();
    scores[handle] = score;
    emit ScoreUpdated(handle, score);
}

function finalizeWeightTournament(string calldata winnerHandle)
    external onlyOracle onlyActive nonReentrant
{
    if (kind != Kind.ContributionWeight) revert WrongKind();
    address payable winner = githubToWallet[winnerHandle];
    if (winner == address(0)) revert UnregisteredHandle();
    /* same effects, event, and call as settleMilestoneWinner */
}
```

Listing 5.5: Weight mode - updateScore and finalizeWeightTournament (excerpt; the last lines are abbreviated)

Figure 5.2 shows the Weight-mode flow as implemented in the contract. The oracle script does not drive this flow (Table 4.1), so the figure describes the contract behavior only.

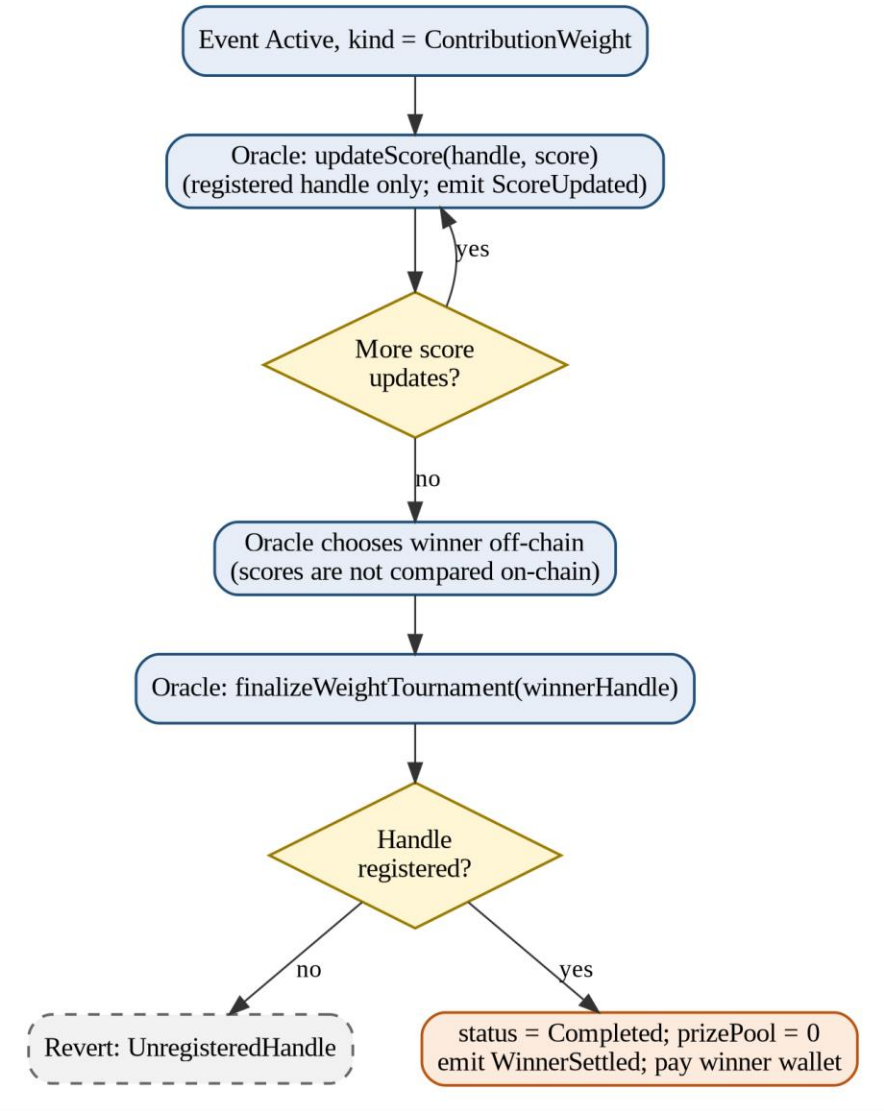


Figure 5.2: Contribution Weight flow in MergeXHub (contract behavior).

5.3.6. Refund After Deadline

refund lets only the host reclaim the prize pool, only while the event is Active, and only when block.timestamp is at or after the deadline (Listing 5.6). It sets the status to Completed and the pool to zero before sending the ETH to the host. Since the check is on Active status, a host that never funds has nothing to refund, and a host who funds after the deadline can refund immediately.

```
function refund() external onlyHost onlyActive nonReentrant {
    if (block.timestamp < deadline) revert DeadlineNotReached();

    uint256 amount = prizePool;
    status    = Status.Completed;
    prizePool = 0;

    emit RefundIssued(msg.sender, amount);

    (bool ok,) = payable(host).call{value: amount}("");
    if (!ok) revert TransferFailed();
}
```

Listing 5.6: Refund - refund (excerpt)

5.3.7. Events and Errors

Five events record the lifecycle: DeveloperRegistered(handle, wallet), PrizeFunded(host, amount), ScoreUpdated(handle, score), WinnerSettled(handle, wallet, amount), and RefundIssued(host, amount). None of the parameters is indexed, so filtering by handle requires reading the log data. The twelve custom errors (NotHost, NotOracle, NotSetup, NotActive, WrongKind, AlreadyFunded, ZeroValue, HandleTaken, WalletTaken, UnregisteredHandle, DeadlineNotReached, TransferFailed) give each failed check a distinct, cheap revert reason. No API or interface reads these events in the repository.

In summary, the excerpts show role checks, status checks, and a safe payout order in the code, but the repository's test results were not run or reported for this document (Section 5.6), so no claim of correctness or security beyond reading the source is made.

5.4. Main Workflows

Figure 5.3 gives an overview of the life of one Milestone Race event; the refund path is drawn separately in Figure 5.4.

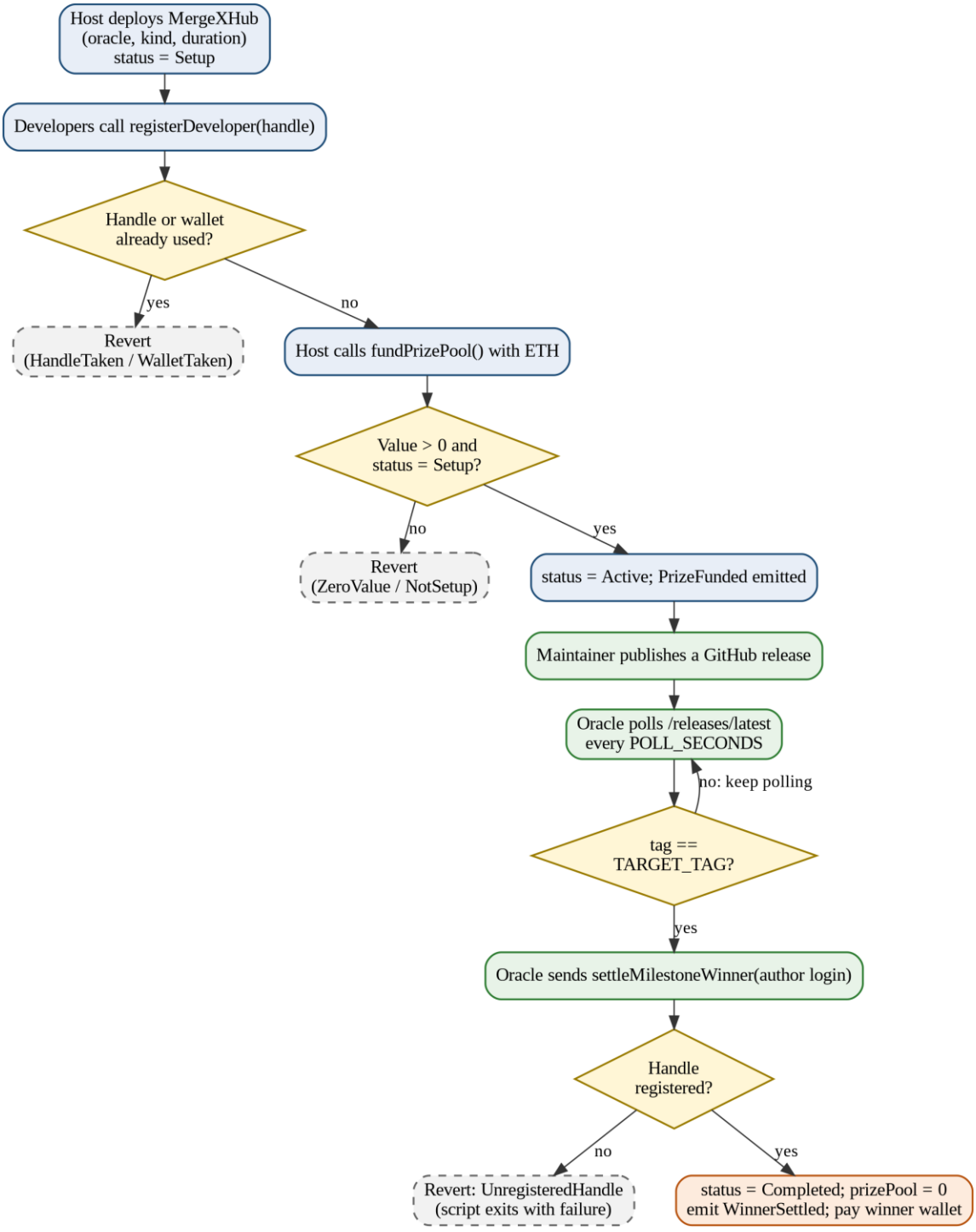


Figure 5.3: Main workflow of the system (Milestone Race).

5.4.1. Setup and Registration

1. The host deploys MergeXHub with the oracle address, the mode (Kind), and a duration in seconds. The deployer becomes the host, the deadline is fixed, and the status is Setup.

2. Each developer calls registerDeveloper with their GitHub login from the wallet that should receive the prize. A handle or a wallet can be registered once.

5.4.2. Funding

1. The host calls fundPrizePool with ETH. The amount must be non-zero.
2. The status becomes Active and registration closes. PrizeFunded is emitted.

5.4.3. Race Settlement

1. A maintainer publishes a release on the target repository.
2. The oracle polls the releases endpoint and compares the latest tag with the target tag.
3. On a match it signs and sends settleMilestoneWinner with the release author's login.
4. The contract checks the caller, status, mode, and registration, updates its state, emits WinnerSettled, and pays the registered wallet in the same transaction.

The winner is the author of the latest release, which is not necessarily the developer who wrote the code. The system trusts the repository's release process.

5.4.4. Refund

If the event is still Active at or after the deadline, the host calls refund and receives the full prize pool. If the event was already Completed by settlement or refund, the call reverts with NotActive.

Figure 5.4 shows the checks applied to a refund call.

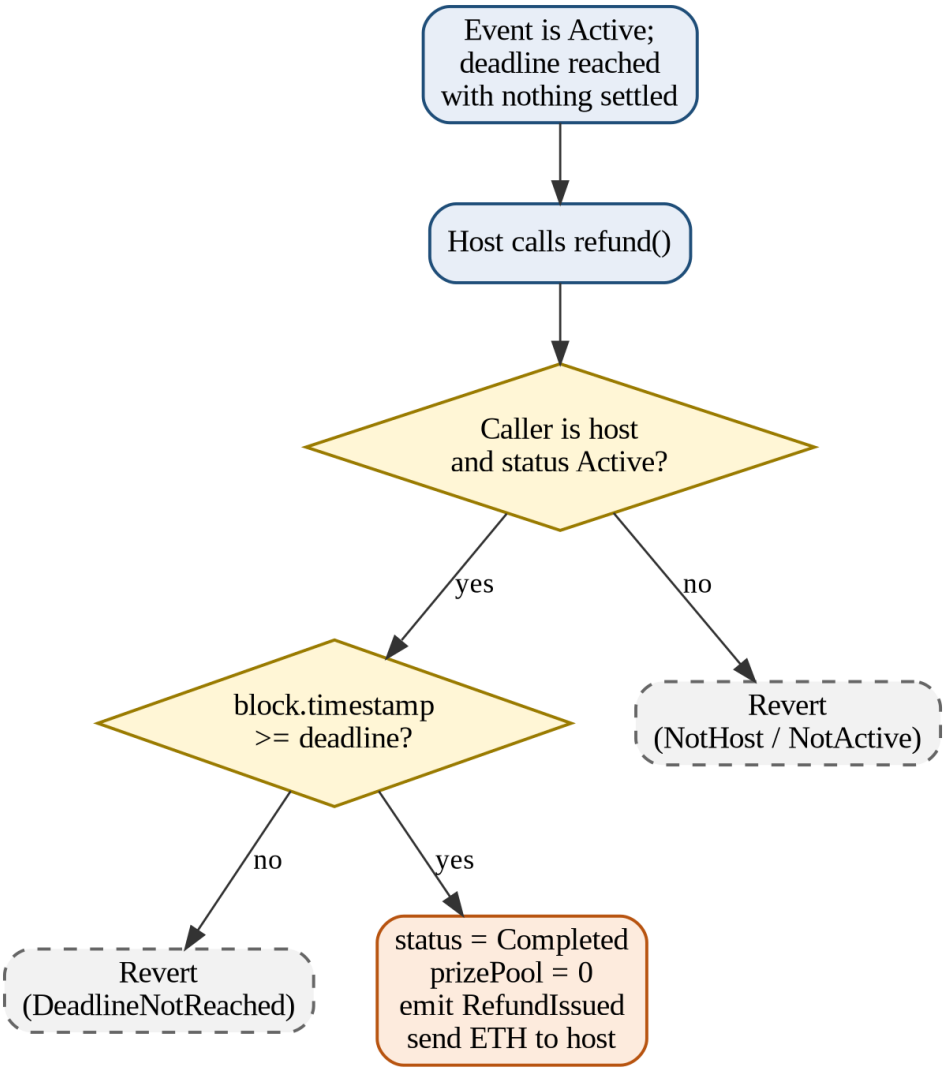


Figure 5.4: Refund flow of MergeXHub.

5.5. Deployment and Demonstration

The README gives the quickstart: clone the repository, install the oracle dependencies with pip install web3 requests, compile with forge build, run the tests with forge test, and run bash demo.sh for an end-to-end demonstration on a local Anvil node. The prerequisites are Foundry (forge and anvil) and Python 3.10 or later. A project video is linked in Appendix A.3 [11]. This report does not reproduce the demonstration's output, and the way the host deploys the contract inside the script was not part of the files read for this report. There is no frontend: the host, developers, and oracle all interact through transactions and scripts.

5.6. Testing

The repository contains a test directory for the Foundry suite and the README lists forge test as part of the quickstart. The design document lists intended checks such as non-oracle settlement

reverting, unregistered winners reverting, double settlement reverting, refund before the deadline reverting, and a reentrancy attempt being blocked. The presence of these files documents intended checks; it does not establish that they passed. No test output, coverage report, gas report, fuzzing result, static-analysis result, or independent audit was supplied or generated for this report, so no results are claimed.

6. BENEFITS, CHALLENGES, AND LIMITATIONS

6.1. Benefits

The following benefits are design-level observations about the prototype.

- **Visible, locked prize:** the host's ETH is held by the contract, so developers can check that the funds exist before the event starts.
- **Automatic settlement:** a matching release tag leads to a payout in a single transaction, with no manual release by the sponsor.
- **Write-once registry:** a handle and a wallet can each be registered only once, and only during Setup, which stops overwriting of existing bindings.
- **Sponsor protection:** the host can reclaim the pool after the deadline if the event is still Active.
- **Safer payout code:** state is updated before ETH is sent, payout functions use nonReentrant, and failed transfers revert instead of silently losing state.
- **Traceability:** five events and twelve custom errors make the lifecycle and failed actions easy to follow.
- **Educational value:** the project connects GitHub data, an off-chain script, and a smart contract, and shows where a blockchain helps and where trust in an off-chain component remains.

No user study was performed, so the report does not claim improved outcomes, reduced costs, or production-grade security.

6.2. Challenges and Limitations

- **Single trusted oracle:** one address decides every settlement and the contract does not re-check GitHub. A compromised or dishonest oracle can send the prize to any registered handle.
- **No oracle key rotation:** the oracle address is immutable and the script reads its private key from an environment variable, so a leaked key cannot be replaced in a deployed contract.
- **Unverified identity:** nothing proves that the wallet owner controls the GitHub handle, so a handle can be registered by someone else before its real owner registers. The contract also accepts an empty handle.
- **Release author is not the contributor:** the winner is the author of the latest release, who may be a maintainer rather than the developer who did the work.
- **Narrow evidence:** the oracle checks only the latest release. A newer release can hide the target release, and a login differing only in letter case will not match the registered handle.

- **Weight mode is incomplete:** the contract pays whichever registered handle the oracle names without comparing scores, scores can be overwritten freely, and the oracle script does not implement this mode.
- **Deadline semantics:** the deadline gates only refund. Settlement is still possible after the deadline until the host refunds, and a host can fund late and refund at once. The host also chooses when registration closes by funding.
- **Stuck winner wallet:** if the winner's wallet rejects ETH, settlement reverts, so the prize remains locked until the host can refund after the deadline.
- **Single winner:** there is no prize splitting, tie handling, or partial payout.
- **Public metadata:** handles, wallet addresses, and scores are readable by anyone, linking a GitHub identity to an address.
- **Local environment only:** the demonstration targets a local Anvil chain. No public testnet or mainnet deployment is evidenced.
- **No evaluation:** no test results, gas measurements, fuzzing, static analysis, threat model, or independent audit are supplied. The prototype must not be described as production-ready or audited, and it should be used only with test funds.

6.3. Issues from the Design Document and Their Status

The project's design document lists ten issues found in the earlier prototype. Table 6.1 compares them with the current MergeXHub.sol, based on reading the source and not on test results.

Table 6.1: Design-document issues and their status in MergeXHub.sol

#	Earlier issue	Status in MergeXHub.sol
1	Handle squatting and overwriting	Partly addressed: write-once and Setup only; ownership still unverified
2	No refund path	Addressed: deadline and refund()
3	Leader tracking bug	Addressed by removing on-chain leader tracking; the oracle names the winner, so trust shifts to the oracle
4	transfer() gas limit and reentrancy	Addressed: call with checks-effects-interactions and nonReentrant
5	Repeated or anyone's funding	Addressed: onlyHost, onlySetup, ZeroValue
6	Single oracle	Open
7	No ties or prize splitting	Open
8	Missing events	Addressed: five events
9	No Setup check in registration	Addressed: onlySetup
10	Oracle key management	Open: immutable oracle, key from environment variable

6.4. Improvement Priorities

Table 6.2: Priorities for improving the system

Priority	Practical improvement
Verify identity	Link handle and wallet with a wallet-signed message published in a GitHub gist, checked by the oracle, and reject empty handles
Reduce oracle trust	Use an m-of-n oracle set or an optimistic settlement with a dispute window; add a timelocked oracle rotation
Complete Weight mode	Implement scoring and score pushes in the oracle, and make the contract or a verifiable snapshot decide the winner; add ranked payouts and tie rules
Tighten deadline rules	Decide whether settlement should stop at the deadline and whether the host may fund after it
Make the oracle robust	Check all releases for the target tag, compare logins case-insensitively, retry failed transactions, and keep the key in a secrets manager
Validate reliability and security	Run forge test with coverage and fuzzing in a recorded environment, add static analysis, and obtain an independent review
Assess real-world fit	Add a frontend, run a testnet pilot, and evaluate cost, governance, and legal requirements before any real-money use

7. CONCLUSION AND FUTURE WORK

The project demonstrates a hybrid approach to paying for open-source work. A host locks ETH in MergeXHub, developers bind their GitHub handles to wallets during Setup, and a Python oracle that watches the GitHub releases endpoint triggers an atomic payout to the author of the target release. The host can reclaim the funds after the deadline. The contract uses custom errors, events, nonReentrant, and a safe payout order, and it removes several problems that the earlier prototypes had.

The implementation also shows that a smart contract does not remove the need for trust: the oracle decides the winner, the host picks the oracle, and nothing verifies that a wallet owner controls the GitHub handle they registered. The Weight mode is only partly implemented, the system runs on a local chain, no test results are reported, and it has not been measured or audited. It is not production-ready.

Future work should include:

- verified identity linking with signed gists and rejection of empty handles;
- a multi-oracle or dispute-window design and oracle key rotation;
- a complete Weight mode with scoring in the oracle, ranked payouts, and tie rules;
- a more robust oracle that scans all releases, normalizes logins, and retries transactions;
- recorded tests, fuzzing, static analysis, gas measurement, and an independent review;
- a frontend dashboard with a live leaderboard and event history; and
- a public testnet pilot, with all evaluation using test funds until the trust model and legal requirements are assessed.

8. SDGs ADDRESSED

The United Nations Sustainable Development Goals provide broad goals for work, infrastructure, and institutions [10]. The project is an educational prototype, so the links below describe potential relevance, not measured impact.

Table 8.1: Mapping of the Project to the Sustainable Development Goals

SDG	Full name of the goal	Official goal statement (United Nations)	Related project feature
SDG 8	Decent Work and Economic Growth	Promote sustained, inclusive and sustainable economic growth, full and productive employment and decent work for all.	Escrowed prize pool, automatic payout to the developer's registered wallet, and host refund after the deadline.
SDG 9	Industry, Innovation and Infrastructure	Build resilient infrastructure, promote inclusive and sustainable industrialization and foster innovation.	Open digital infrastructure combining the GitHub API, an off-chain oracle, and a smart contract on an EVM chain.
SDG 16	Peace, Justice and Strong Institutions	Promote peaceful and inclusive societies for sustainable development, provide access to justice for all and build effective, accountable and inclusive institutions at all levels.	Public payout rules in code, five lifecycle events, and custom errors that make payouts auditable.

8.1. SDG 8: Decent Work and Economic Growth

A transparent escrow could make it easier for independent developers to be paid for open-source work without relying on a custodian. The project has not been tested with real users or real payments, and it reports no effect on income or employment.

8.2. SDG 9: Industry, Innovation and Infrastructure

The project is an educational exploration of open digital infrastructure that combines a public API, an off-chain service, and a smart contract. The local single-node setup demonstrates a concept, not resilient or production infrastructure.

8.3. SDG 16: Peace, Justice and Strong Institutions

Public payout rules and contract events could support accountable, auditable prize distribution compared with opaque judging. Because one oracle decides the outcome and no dashboard reads the events, the contribution is exploratory and limited.

9. REFERENCES AND LINKS

[1] S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008. [Online]. Available: https://bitcoin.org/bitcoin.pdf. Accessed: Oct. 26, 2026.

[2] G. Wood, "Ethereum: A Secure Decentralised Generalised Transaction Ledger (Yellow Paper)." [Online]. Available: https://ethereum.github.io/yellowpaper/paper.pdf. Accessed: Oct. 26, 2026.

[3] Solidity Team, "Solidity Documentation." [Online]. Available: https://docs.soliditylang.org/. Accessed: Oct. 26, 2026.

[4] Ethereum Foundation, "Oracles," ethereum.org developer documentation. [Online]. Available: https://ethereum.org/en/developers/docs/oracles/. Accessed: Oct. 26, 2026.

[5] OpenZeppelin, "OpenZeppelin Contracts Documentation (ReentrancyGuard)." [Online]. Available: https://docs.openzeppelin.com/contracts/. Accessed: Oct. 26, 2026.

[6] Foundry, "The Foundry Book (forge, anvil)." [Online]. Available: https://book.getfoundry.sh/. Accessed: Oct. 26, 2026.

[7] web3.py, "web3.py Documentation." [Online]. Available: https://web3py.readthedocs.io/. Accessed: Oct. 26, 2026.

[8] GitHub, "REST API Documentation: Releases." [Online]. Available: https://docs.github.com/en/rest/releases. Accessed: Oct. 26, 2026.

[9] Project source repository, GitHub. [Online]. Available: https://github.com/lakshmanbhukya/MergeX. Accessed: Oct. 26, 2026.

[10] United Nations, "The 17 Goals: Sustainable Development." [Online]. Available: https://sdgs.un.org/goals. Accessed: Oct. 26, 2026.

[11] Project video, Google Drive. [Online]. Available: https://drive.google.com/file/d/1nhWr-t4AsQn9ki6BKBSrOrZdTIJhMseA/view?usp=drive_link. Accessed: Oct. 26, 2026.

APPENDIX A

A.1. Project Repository

Project source repository: https://github.com/lakshmanbhukya/MergeX.git

Project page linked from the repository: https://lakshmanbhukya.github.io/MergeX/

A.2. Repository QR Code



Figure A.1: QR code for the project source repository.

A.3. Project Video

The project video is available at: https://drive.google.com/file/d/1nhWr-t4AsQn9ki6BKBSrOrZdTIJhMseA/view?usp=drive_link



Figure A.2: QR code for the project video.

A.4. Source Files Referenced in Chapters 4 and 5

The excerpts and workflow descriptions refer to these repository files: src/MergeXHub.sol, oracle/oracle.py, test/ (Foundry tests), demo.sh, mock/repos/demo/demo/releases, foundry.toml, foundry.lock, .gitmodules, README.md, and the design document.

A.5. Demonstration Safety

Use test networks or a local Anvil chain with test funds only. Never commit the oracle private key or any other secret to the repository, keep it in an environment variable or secrets manager, and do not treat the local setup as a production environment.